EECS 470

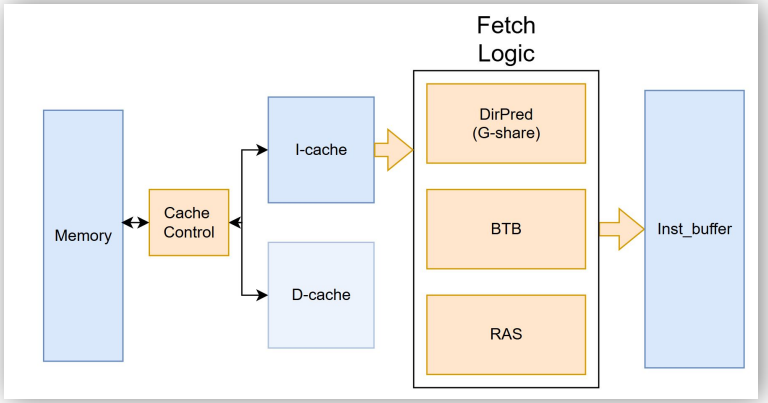
Final Presentation

Group 10
Zen Lake Pro Max

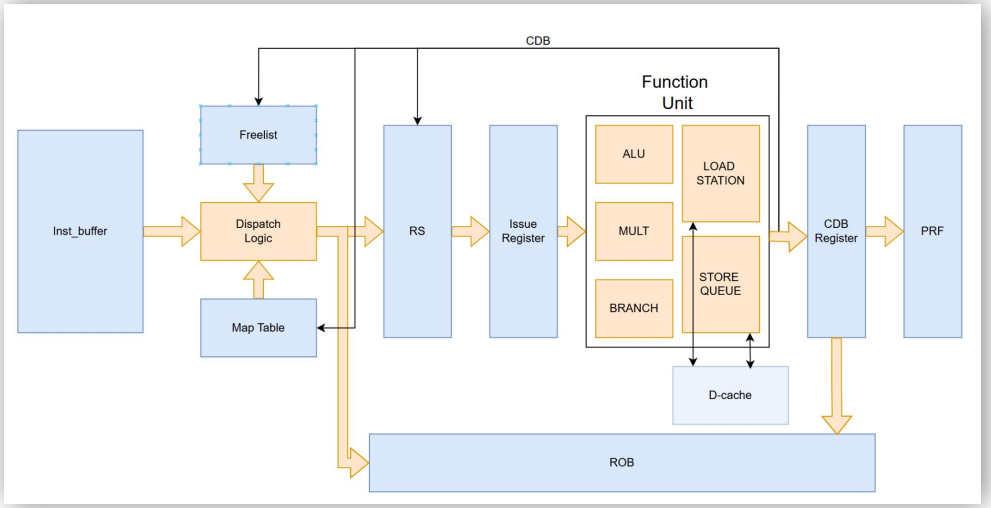
Jiahe Ding, Hongzheng Hu, Qijia Liu,
Junde Song, Junwen Yu, Kangqi Zhang

Overall Design

- R10K
- 6-stage pipelined



Frontend modules

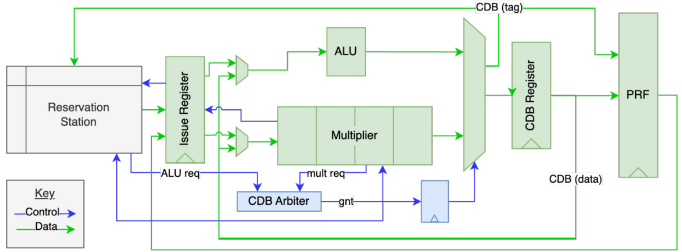


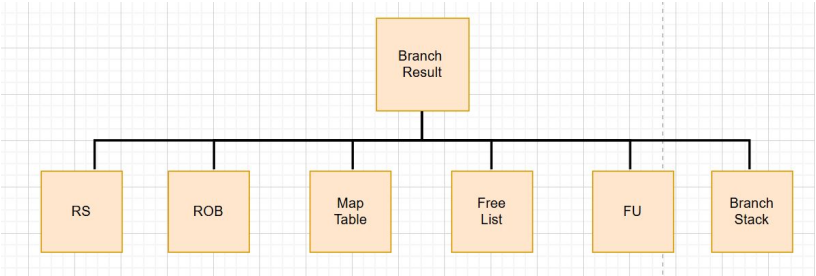
Core modules

Core Features

- N-way Superscalar Execution
  - Instantialized as N=3
- Early Tag Broadcast
- Early Branch Resolution

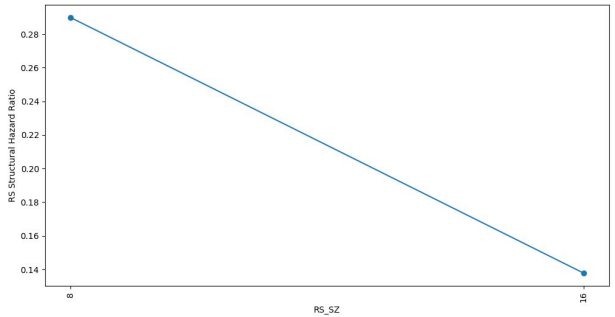
```
`define FETCH_WIDTH 4
`define DISPATCH_WIDTH 3
`define CDB_WIDTH 3
`define RETIRE_WIDTH 3
```

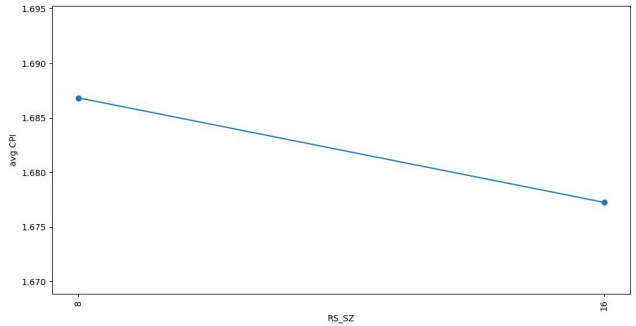




RS Size Optimization Summary

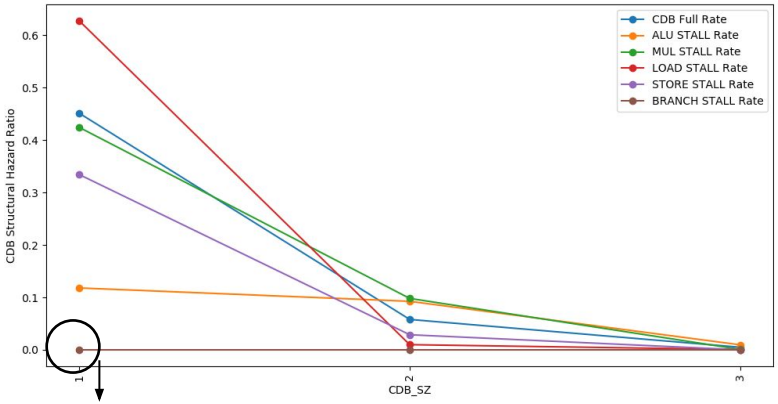
- Originally, RS Size = 8, leading to more than **28%** RS Structural Hazard.
- Increase RS Size to 16, RS Structural Hazard drop by more than **50%**
- CPI improves by **~0.5%**



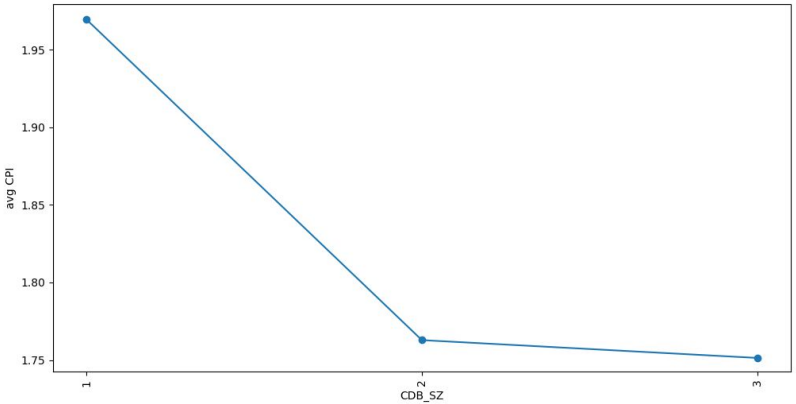


CDB Width Optimization Summary

- Originally, CDB width= 1, causing **>45%** CDB stalls.
- Increase CDB width by 1, decreases CDB Structural Hazard by > **8** times
- CPI improves by **~10%**



Prioritize Branch Insns



Cache

- Separate buffers for requests
- Stream buffer
- MSHR

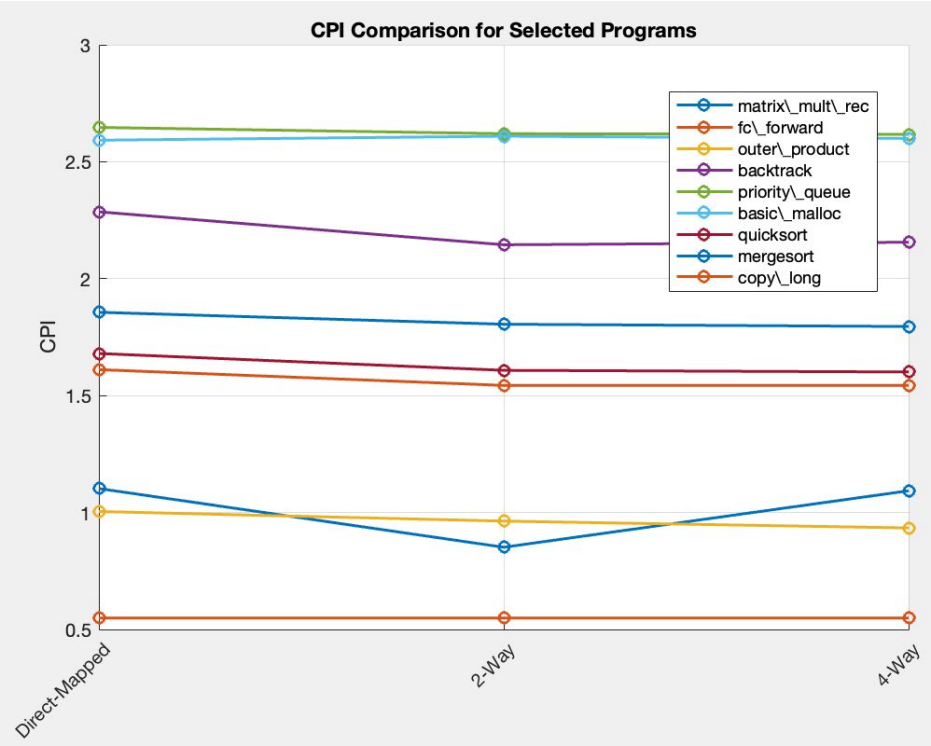
Stream Buffer

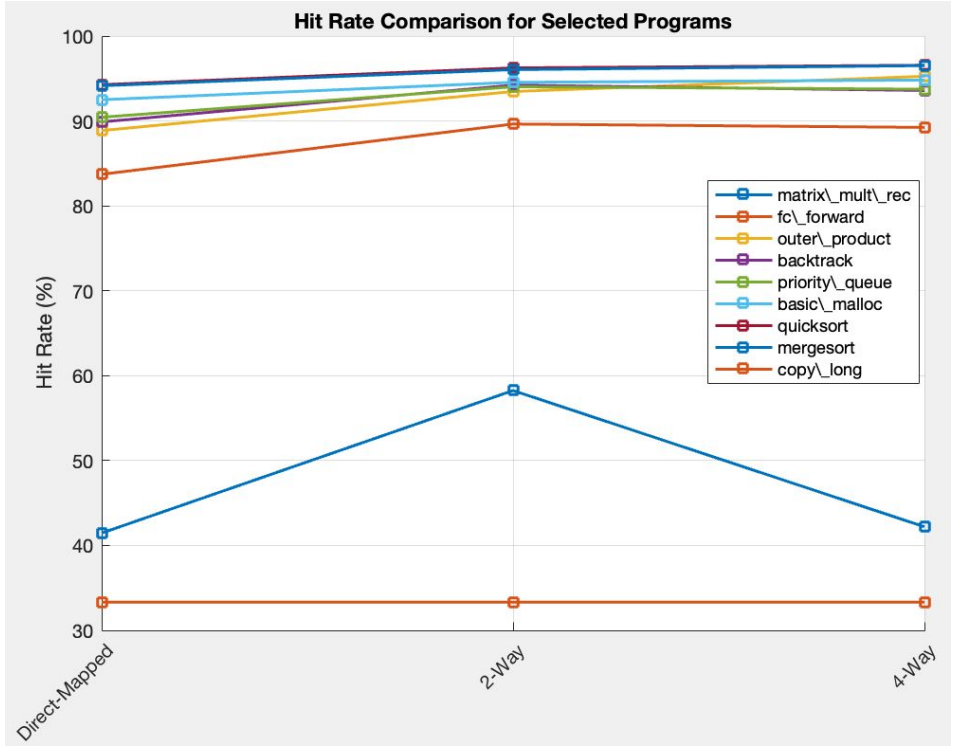
- Prefetch
- Active mem req
- Write upon hit
- Clear upon miss

Cache

- Separate buffers for requests
- MSHR
  - Non-blocking
  - Passive mem req
  - Broadcast to Load Station

Cache Optimization





More Ways!=Better Performance

Direct map ➔ 2 way

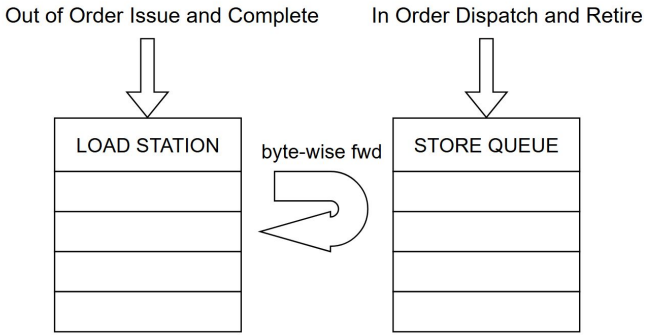
- Hit rates increase by 3–10 %
- CPI dropped accordingly

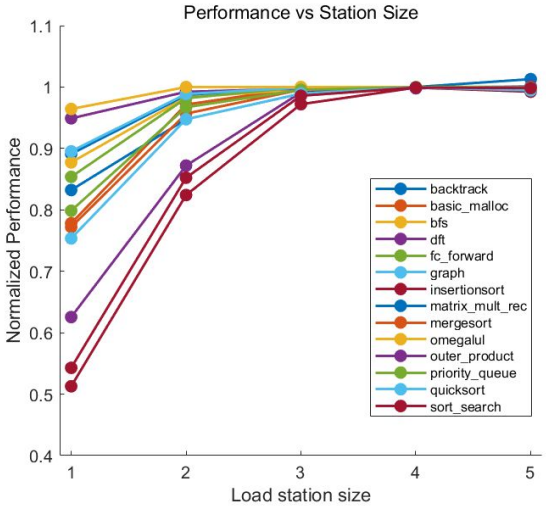
2 way ➔ 4 way

- Very little additional improvement!
- Data paths & Violations

Load Station

- Motivation
  - Memory latency is long!
- So, we build a load station
  - Leverage Non-blocking cache
  - Multi load waiting for memory





Branch Predictor

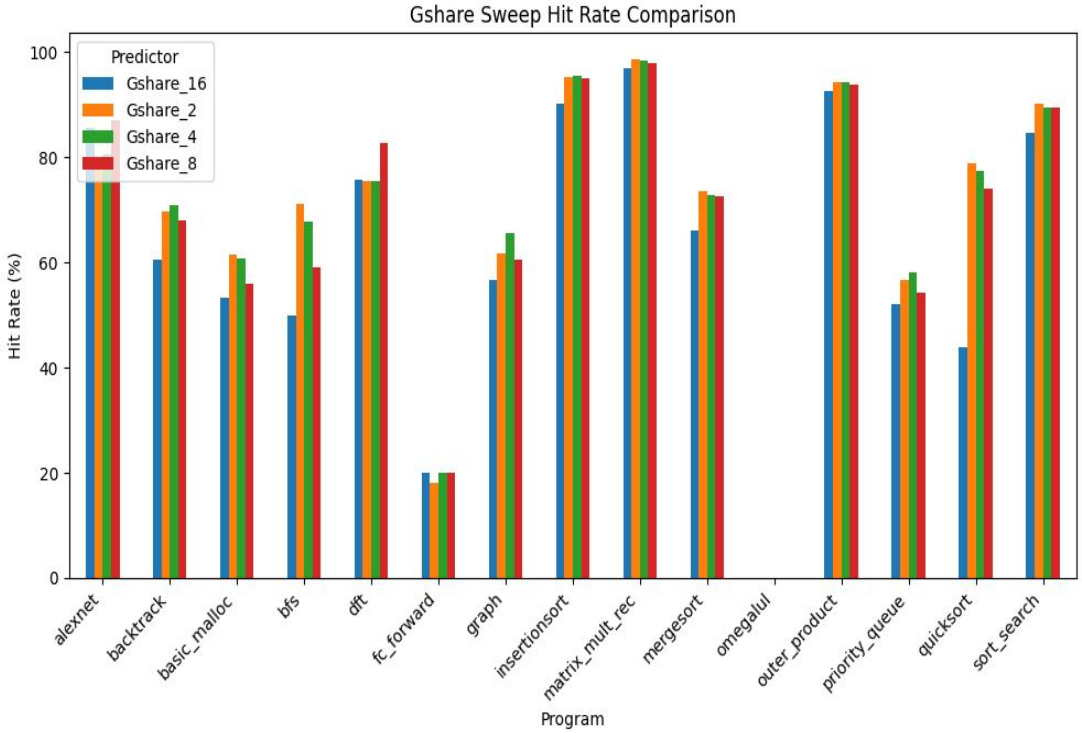
We implemented:

- Gshare with BHR sweep (2,4,8,16) 💙
- TAGE, but find negative result😅
- Use 8-BHR Gshare as benchmark

Have fun with:

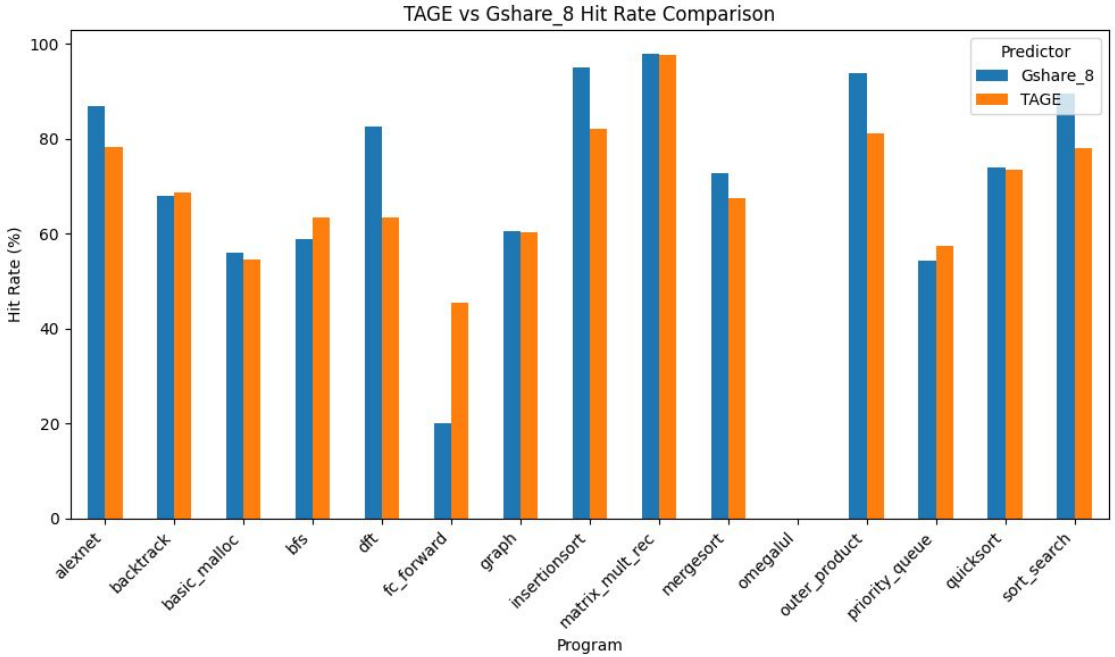
- Dynamic perceptron predictor(surprisingly good) 👍,
- Static 1-layer CNN predictor 🤩
- Static N-layer CNN predictor (~~useless~~ but cool)😎

Gshare Sweep Result



- Hit rate varies a lot with different programs
- Smaller programs is not friendly to large BHR G-shares due to warming up

TAGE Result

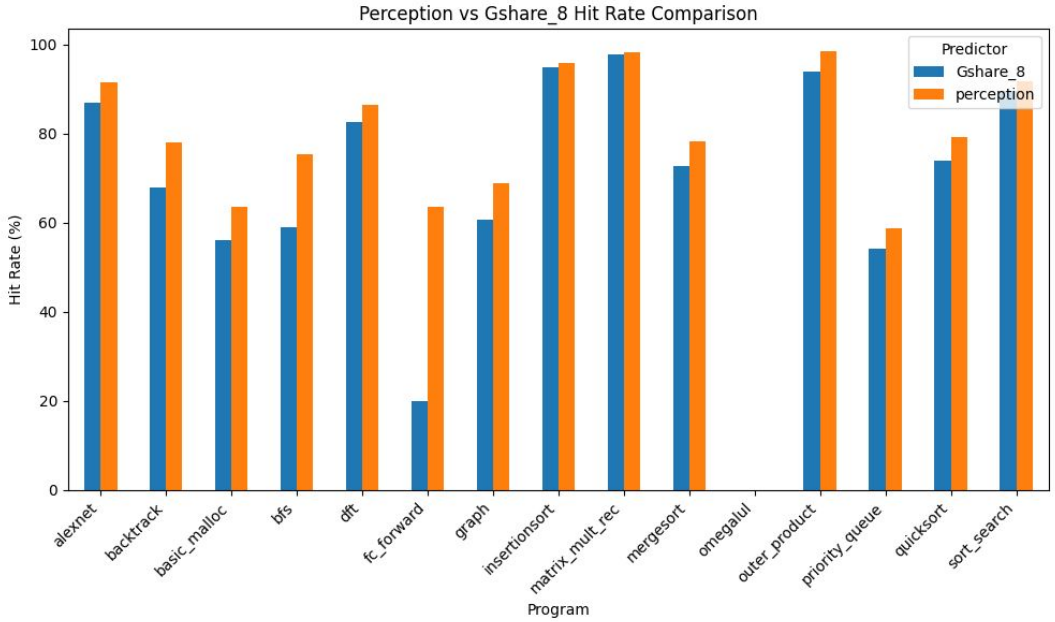


Each pht entry has:

- 2-bits saturated counter
- 1-bit useful bit : set 1 if predicted right last round
- valid bit
- tag, different from pht index tag
- large area, much more bits than Gshare

Useful && tag match -> candidate
If candidate, Gshare_8 > Gshare_4 > Gshare_2 > Simple predictor

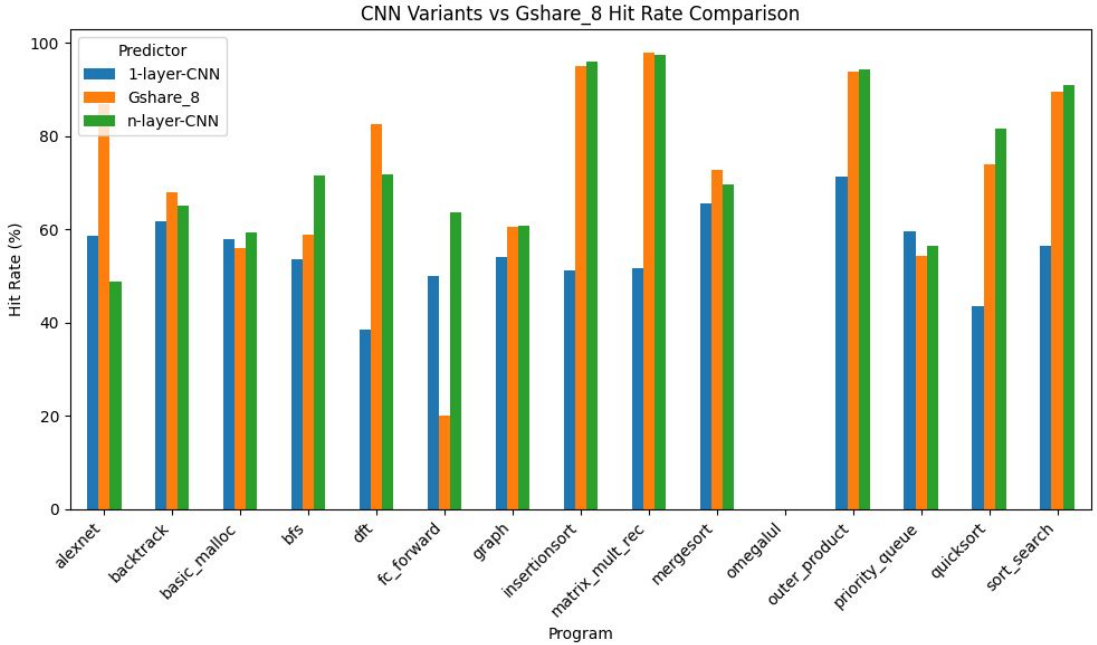
Perceptron Result



- Dynamic, weight updates during computing
- Choose BHR=20
- Hard to improve (maybe?)

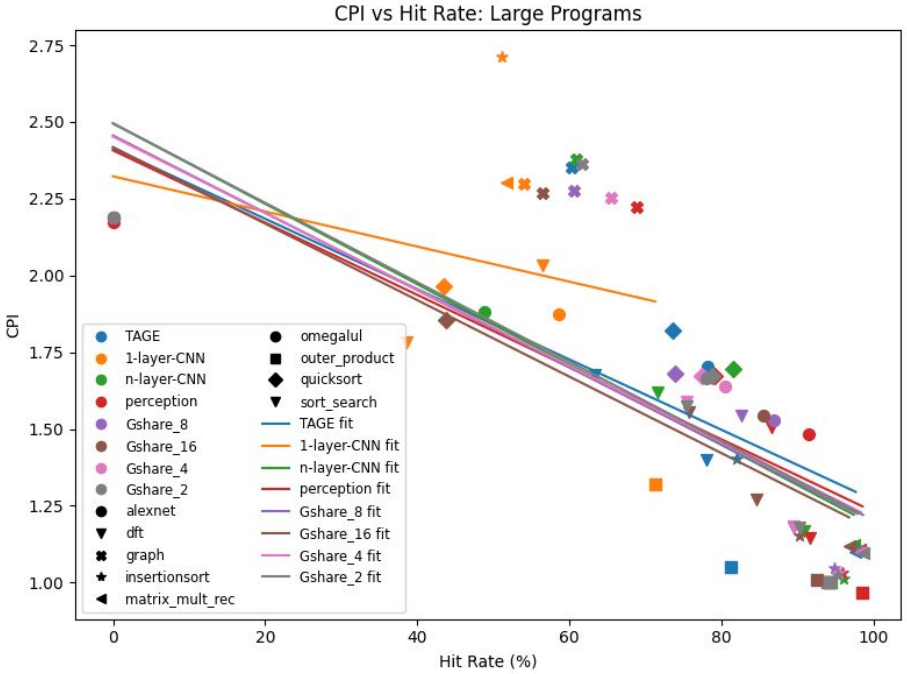
$$\text{pred\_taken} = \sum_{i=0}^{N-1} w_i \, \text{bhr}[i] \;\geq\; 0.$$

CNN Result



- use kernel to extract specific pattern, such as [1,-1,1]
- offline training, only parameters input
- since static, friendly to pipeline (multi-stage branch predict)
- multilayer CNN can extract more complex feature(not consecutive, like[-1,x,x,1,1])

Interesting Statistics



relation between CPI and hit rate large programs

- Same data point shape means same programs, same data point color means same predictor
- Different branch predictors have similar relation between hit rate and CPI, even if with large deviation.

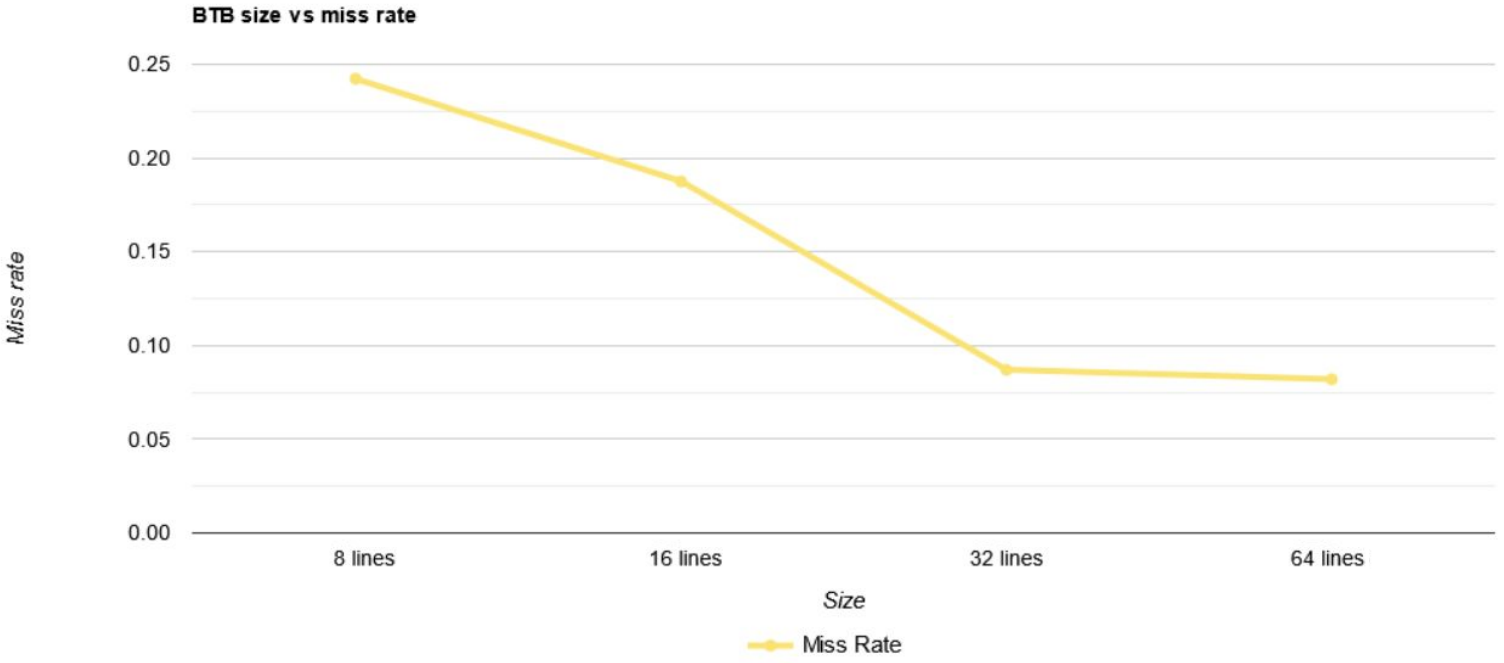
$$CPI = 2.45 - 0.0125(Hit\ rate)$$

Optimization Attempt of BTB

Direct mapped vs Four way associative

- Direct mapped miss rate : 0.1876
- 4-way associative: 0.0686
- CPI looks similar on short programs(except for sort_search 1.935 vs 1.215)

Direct mapped btb size and miss rate



Round Robin vs LRU

- No apparent differences on short programs.

Hot loading

- Intuition: Borrow ideas from warming up.
- Construct a btb that overlap with the historical btbs most
- Miss rate: 0.0647(a bit better)

Connect to Branch history

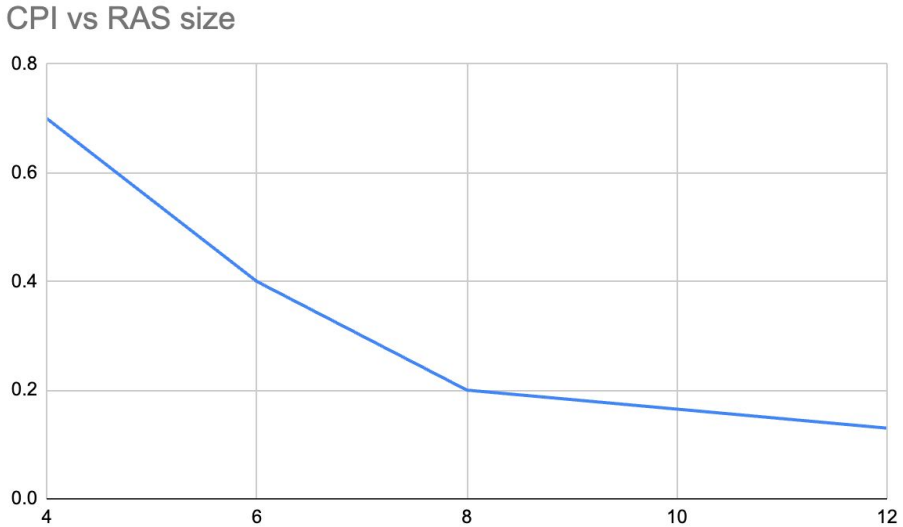
- Intuition: Same address may have multiple target(function pointer)
- Tried three bits and two bits branch history(for fast warm up) with direct mapped BTB. Doesn’t show improvement.

RAS

- Function return jumps to where it was called
- Store address at func call, pop when return
- Very brittle: one mistake messes up all subsequent predictions
  - Need ebr checkpoint
  - Need to address structural hazard

RAS

- Choice of size: monitor structural hazard
- 4 -> 8: 1.5% CPI improvement on average



Issue In Program Order

- CDB/FU width is restricted
- Heuristic: issue older instructions first

Other priority selectors

- Psel_gen alternates between MSB and LSB

  - alt dispatch + seq issue: 0.03%
  - seq dispatch + seq issue: 0.09%

Age matrix

- [rs_size][rs_size] age;
- age[i][j]: entry[i] is younger than entry[j]

Age matrix

- Performance:
  alexnet: 0.4%
  quicksort: 2%
  sort_search: 5%
  outer_product: 6%

- More useful when more CDB/FU structural hazard

Clock Period

- 10ns
- Have to cut some forwarding if 9.2ns, overall less performance
- Critical Path
  - rob requests to retire
  - sq request to retire a store
  - dcache responds
  - rob returns old tags to freelist

C++ simulator

```
// complete
        rs.onCdbBc(cdb.getEarlyCDBTags());
        rs.onBrRes(cdb.getBrResPacket());
        dispatcher.onBrRes(cdb.getBrResPacket());
        rob.recCDBEarlyTags(cdb.getEarlyCDBTags());
        fetcher.onBrRes(cdb.getBrResPacket(), cdb.getPredictionResPacket());
        /* Early Branch Resolution: selective squash */
        issueReg.onBrRes(cdb.getBrResPacket());
        fu.onBrRes(cdb.getBrResPacket());
        /* Early Branch Resolution: update mt tag ready*/
        mt.recCDBTagMasks(cdb.getEarlyCDBTags(), cdb.getCdbBmasks());
        /* Early Branch Resolution: restore copies */
        mt.onBrRes(cdb.getBrResPacket());
        fl.onBrRes(cdb.getBrResPacket());
```

C++ simulator

- Pros:
  - Help a lot with debugging (breakpoint, state tracking, etc.)
  - Expose issues that don’t cause observable problems in test programs
- Cons:
  - Still a lot to code
  - Still not an easy task to debug
  - No natural way to represent hardware logic (e.g., wire)

Interface

```
// ------------------------ cache signals ------------------------
    ADDR                                                    icache_mem_addr;
    MEM_COMMAND                                             icache_mem_command;
    ADDR                                                    dcache_mem_addr;
    MEM_COMMAND                                             dcache_mem_command;
    MEM_BLOCK                                               dcache_mem_write_data;
```

```
modport dcache(
    input  transac_tag_dcache,
    input  mem_read_data,
    input  mem_read_data_tag,
    output dcache_mem_addr,
    output dcache_mem_command,
    output dcache_mem_write_data,
```

```
dcache data_cache (
    .sys(cpuBus.sys_in),
    .bus(cpuBus.dcache),
```

Takeaway

- Think in terms of actual hardware when coding
- Verification is more important (and also much more work) than we initially thought

Thank You!
Questions?

Group 10
Zen Lake Pro Max